

Síntesis Musical

Procesamiento de Audio y Síntesis de Instrumentos

Introducción

Deberán desarrollar una aplicación de escritorio capaz de sintetizar instrumentos a partir de un archivo MIDI, aplicar distintos efectos musicales, y analizar el resultado en el dominio de la frecuencia.

Tienen libertad de implementar la interfaz gráfica usando el lenguaje y framework que deseen. Recomendamos usar Python con las bibliotecas PyQt, PyAudio, SciPy y NumPy.

Deberán entregar los testbenches utilizados para validar el funcionamiento de los distintos algoritmos.

MIDI (Musical Instrument Digital Interface)

MIDI consiste en un estándar de comunicación entre dispositivos, usualmente uno siendo un instrumento musical. Permite, por ejemplo, comunicar un piano digital con una computadora o un sintetizador. Es importante aclarar que el protocolo MIDI no involucra audio, sino que utiliza *eventos* para comunicar lo ocurrido en el dispositivo. Los archivos MIDI permiten almacenar dichos eventos para acceder a ellos más tarde.

Los archivos MIDI se componen de uno o más pistas (“tracks”). A su vez, cada “track” se compone de una lista de eventos, que pueden sintetizarse mediante distintos métodos para producir una grabación de audio.

Cada instrumento (o “programa”, en la terminología MIDI) es asignado a un “canal” numerado del 1 al 16. Generalmente los eventos de una misma pista se corresponden con un sólo “canal”, pero es posible que la pista contenga eventos asociados a diferentes “canales”.

Los principales tipos de eventos son:

- Inicio de nota: Por ejemplo, cuando se presiona la tecla en un piano o una cuerda de guitarra.
- Fin de nota: Por ejemplo, cuando se suelta la tecla o cuerda.
- Configuración de *tempo*: Evento “meta” que configura el *tempo* de la canción.
- Cambio de “programa” (instrumento).

Observaciones:

- Modificar el *tempo* afecta el intervalo de tiempo entre eventos, pero no modifica el *pitch* de las notas.

- Cada evento incluye una marca temporal que indica el intervalo de tiempo transcurrido desde el evento anterior. Ya que el *tempo* de la pista puede modificarse, los eventos no tienen una marca temporal en segundos sino en “ticks”, y la configuración del *tempo* establece la relación entre “ticks” y segundos.
- La duración de una nota no es lo mismo que la duración de un sonido. Muchas veces, el sonido se extiende durante más tiempo que lo que dura la nota. Por el contrario, si se toca *staccato*, el sonido durará menos. Si tienen esto en cuenta, la calidad de la síntesis mejorará considerablemente.

Objetivos del proyecto

La aplicación debe ser capaz de:

- Abrir y decodificar un archivo en formato MIDI.
- Asignar cualquier instrumento a cualquier canal.
- Sintetizar cada pista en una grabación de audio en función de los instrumentos asignados.
- Agregar y configurar diferentes efectos de audio para cada pista, y también para la mezcla total.
- Generar un espectrograma a partir del audio generado (ver el punto 1.2).
- Guardar el audio generado en algún formato estándar (wav, mp3, etc).

Una vez finalizada la aplicación, ensayarla con un fragmento de longitud considerable del segundo movimiento (Adagio) del Concierto de Aranjuez, de Joaquín Rodrigo, a partir de un archivo MIDI con la obra. Incluir el archivo de audio generado como parte de la entrega.

Aclaraciones importantes

El foco de este trabajo está puesto en el análisis y procesamiento de señales digitales. El código desarrollado para la implementación de filtros, efectos, síntesis y análisis espectral debe ser de **autoría propia** por parte de los estudiantes, y deben ser capaces de explicar en detalle el trabajo en una exposición oral.

Está permitido el uso de bibliotecas y código escrito por terceros o por IA generativa para el desarrollo de los siguientes aspectos del proyecto:

- Decodificación de archivos MIDI y codificación de archivos de audio (.mp3 o .wav).
- Cálculo de la FFT (excepto en el ejercicio 1.1).
- Interfaz gráfica y funcionalidades de front-end.

Debe aclararse explícitamente el origen de cada archivo o sección de código que no sea de autoría propia. En caso de dudas, consultar con la cátedra sobre la utilización de bibliotecas o código de terceros.

1 Análisis espectral

1.1 Transformada rápida de Fourier (FFT)

La FFT es un algoritmo que permite computar la transformada discreta de Fourier con una complejidad algorítmica $\mathcal{O}(n \log n)$.

- a) Implementar una FFT mediante el algoritmo de Cooley-Tukey en el lenguaje C, usando señales de entrada y salida representadas como *float complex*¹.

Debe poder operar con una cantidad de puntos desde 1 hasta, al menos, 4096, sujeto a las restricciones del algoritmo (potencias de 2). Son libres de implementar el código como prefieran, y externamente deberán presentar la siguiente interfaz:

```
/**
 * @param in    Puntero al buffer con datos de entrada.
 * @param out    Puntero al buffer con datos de salida.
 * @param N      Cantidad de puntos (debe ser potencia de 2).
 */
void fft(float complex *in, float complex *out, size_t N);
```

El buffer de salida puede ser el mismo que el buffer de entrada. La función debe operar correctamente aún en este caso.

- b) Validar el funcionamiento correcto del algoritmo implementando tests funcionales.

Contrastar los resultados del algoritmo FFT con la siguiente implementación de referencia $\mathcal{O}(n^2)$:

```
#include <complex.h>    // float complex, cexpf
#include <stdlib.h>      // malloc, free
#include <stddef.h>      // size_t
#define PI 3.14159265358979323846f

void dft(float complex *in, float complex *out, size_t N){
    float complex *output = (float complex*)malloc(sizeof(float complex)*N);
    for(size_t k = 0; k < N; k++){
        output[k] = 0;
        for(size_t n = 0; n < N; n++){
            output[k] += in[n] * cexpf(-2*PI*I*k*n/N);
        }
    }
    for(size_t k = 0; k < N; k++){
        out[k] = output[k];
    }
    free(output);
}
```

1.2 Espectrograma

Un espectrograma es una imagen tiempo-frecuencia de una señal. Esta imagen permite analizar cómo cambia el espectro de frecuencia de una señal a lo largo del tiempo. Para ello, la señal de entrada se divide en segmentos, y a cada uno de ellos se le aplica la transformada discreta de Fourier (mediante el algoritmo FFT).

¹<https://en.cppreference.com/w/c/numeric/complex.html>

Cuando se aplica la DFT sobre un segmento de la señal de entrada la transformada asume que la señal es periódica, por lo que las discontinuidades entre el inicio y el final del segmento afectarán el resultado incrementando los componentes de alta frecuencia. Para mitigar este efecto, es posible multiplicar el segmento de entrada por una ventana (por ejemplo, la ventana de Hann) la cual atenúa progresivamente las muestras más cercanas al inicio y al final del segmento. La consecuencia de multiplicar las muestras en el tiempo por una ventana, es que el espectro del segmento de entrada se verá convolucionado por el espectro de la función de ventana. Por lo tanto, es importante seleccionar la función de ventana idónea para cada aplicación.

Otra consecuencia de las funciones de ventana es que al dividir la señal de entrada en segmentos sin solapamiento ocurrirá que las muestras cerca de los bordes de los segmentos serán atenuadas y no se verán representados en el espectrograma. Para solucionar este problema usualmente los segmentos sobre los que se computa cada DFT se seleccionan usando cierto porcentaje de solapamiento (*overlap*).

- a) Implementar una función en la aplicación que permita computar y visualizar el espectrograma de la señal de audio de salida. El espectrograma que implementen debe permitir cambiar sus parámetros, como la longitud de cada segmento, el porcentaje de solapamiento y la función de ventana (rectangular, Hann, Hamming, etc.). Está permitido usar una biblioteca específicamente para el cálculo de la FFT (por ejemplo SciPy).
- b) Responder qué relación de compromiso existe al momento de seleccionar:
 - I) Longitud de segmento.
 - II) Porcentaje de solapamiento.
 - III) Función de ventana.
- c) Usando la ventana de Hann, demostrar cuál es el mínimo porcentaje de solapamiento que permite que cada muestra sea ponderada por igual (sumando la ponderación que cada muestra recibe para cada segmento).
- d) **[OPCIONAL]** Obtener el espectrograma de una escala de sol mayor (G3), generada con la síntesis aditiva. Emplear aproximadamente 120 ms por nota ¿Cuales son los parámetros apropiados?

2 Síntesis de instrumentos

Se deben implementar en la aplicación los siguientes métodos de síntesis de instrumentos:

- I) Síntesis basada en muestras (PSOLA).
- II) Síntesis de sonidos mediante modelos físicos (Karplus–Strong).
- III) [OPCIONAL] Síntesis aditiva de sonidos.
- IV) [OPCIONAL] Síntesis de sonidos mediante modulación en frecuencia (FM).

2.1 Síntesis basada en muestras (PSOLA)

La síntesis basada en muestras busca producir sonidos realistas a partir grabaciones reales de instrumentos. Como no es factible tener grabaciones de todas las combinaciones de notas con todas las posibles duraciones, se deben generar las notas a partir de una cantidad reducida de grabaciones, que se pueden estirar y comprimir en tiempo y frecuencia.

Para cambiar la duración de una grabación de audio una primera idea podría consistir en modificar la frecuencia de muestreo a la cual se reproduce. Por ejemplo, si duplicamos la frecuencia de muestreo la duración de la grabación será la mitad. Si efectuáramos un down-sampling con factor de 2 incluso podemos obtener un resultado con igual frecuencia de muestreo pero la mitad de duración. El problema con esta idea es que estamos estirando el espectro y duplicando la frecuencia de todos los componentes presentes en la grabación. Entonces, un requerimiento importante a la hora de estirar o comprimir la duración de una grabación es preservar el contenido espectral y no modificar las frecuencias presentes.

Cabe aclarar que modificar la duración y el “pitch” (la nota, o frecuencia fundamental), son problemas equivalentes. Si logramos implementar un algoritmo que modifica la duración preservando el pitch constante, entonces podemos aplicar un up-sampling o down-sampling para restaurar la duración original de la grabación, obteniendo un fragmento de audio con diferente pitch pero duración constante. Algo similar puede lograrse si se cuenta con un algoritmo que modifica únicamente el pitch, el resultado puede re-muestrearse para restaurar el pitch a su valor original habiendo alterado la duración.

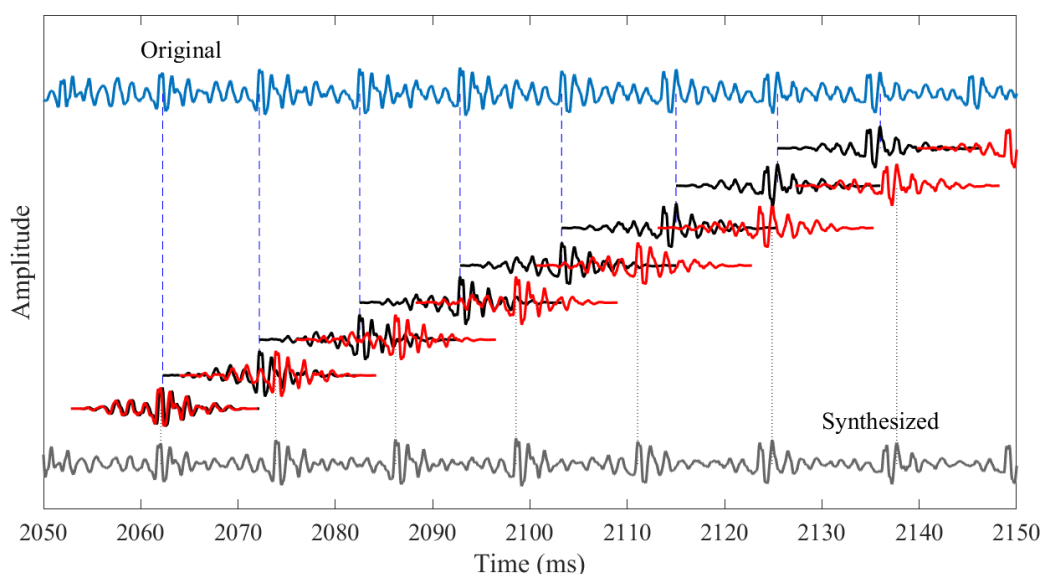


Figura 2: Diagrama conceptual del algoritmo PSOLA de estiramiento en tiempo.

Una de las técnicas más utilizadas para realizar este tipo de transformaciones es el algoritmo PSOLA² (Pitch-Synchronous Overlap and Add), el cual permite modificar de manera independiente la duración y el pitch de una señal de voz o instrumento monofónico, preservando en gran medida sus características perceptuales. El algoritmo PSOLA se basa en segmentar la señal en pequeñas ventanas centradas en instantes de excitación periódica (por ejemplo, los picos asociados al período fundamental en señales cuasi-periódicas). Estas ventanas se reubican en el tiempo y se combinan mediante un esquema de *overlap-add*, permitiendo controlar tanto la duración como el período efectivo de la señal reconstruida. La figura 2 ilustra el “estiramiento” temporal de una grabación.

Deberán implementar un sintetizador musical basado en muestras utilizando PSOLA. Para ello, deberán:

- a) Obtener grabaciones de uno o más instrumentos monofónicos (por ejemplo: guitarra, piano, flauta, etc.), procurando que las notas sean lo más estables posibles en *pitch*.

Analizar las señales para estimar su período fundamental a lo largo del tiempo.

Detectar los instantes de pitch (*pitch marks*), que determinarán los centros de las ventanas a utilizar en el algoritmo.

- b) Implementar el algoritmo PSOLA:

- Segmentación de la señal en ventanas centradas en los *pitch marks*.
- Reubicación temporal de los segmentos según el factor de estiramiento o compresión deseado.
- Reconstrucción de la señal mediante *overlap-add*.

Permitir modificar de manera independiente:

- La duración de la nota (time-stretching).
- El *pitch* (mediante combinación de PSOLA y re-muestreo).

- c) Aplicar el algoritmo implementado a la síntesis musical a partir de archivos MIDI.

- d) Evaluar la calidad del audio generado, analizando:

- i) La preservación del contenido espectral.
- ii) La aparición de artefactos (discontinuidades, efectos metálicos, etc.).
- iii) La influencia de parámetros como el tamaño de ventana y el solapamiento. Experimentar con distintos tipos de ventanas (Hann, Hamming, etc.).

2.2 Síntesis de sonidos mediante modelos físicos (Karplus–Strong)

Modelo de Karplus-Strong

El modelo de Karplus-Strong³ permite sintetizar sonidos similares a instrumentos de cuerda percutida (por ejemplo, guitarra o arpa). Se basa en un sistema discreto que consiste en una línea de retardo realimentada mediante un filtro. En la figura 3 se muestra un diagrama modelo del sistema.

²https://speechprocessingbook.aalto.fi/Representations/Pitch-Synchronous_Overlap-Add_PSOLA.html

³https://ccrma.stanford.edu/~jos/pasp/Karplus_Strong_Algorithm.html

El sistema se excita inicialmente con una secuencia de ruido de longitud finita, que modela la perturbación inicial de la cuerda.

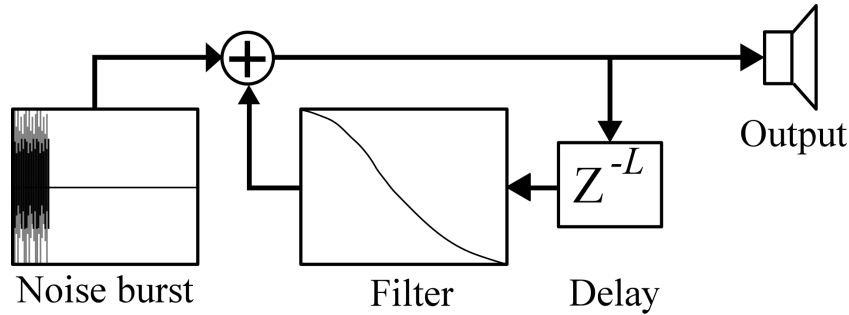


Figura 3: Modelo conceptual de Karplus-Strong

Descripción del modelo

- La línea de retardo de L muestras modela la longitud de la cuerda.
- La realimentación introduce una atenuación progresiva, simulando la disipación de energía.
- El filtro en la realimentación suele ser pasa-bajos, lo cual produce una mayor atenuación de los armónicos de alta frecuencia.

La longitud L determina la frecuencia fundamental de la señal sintetizada. En una implementación típica, esta viene dada aproximadamente por:

$$f_0 \approx \frac{f_s}{L + \tau_g(\omega_0)}$$

donde f_s es la frecuencia de muestreo y $\tau_g(\omega_0)$ es el retardo de grupo del filtro en la frecuencia fundamental.

Implementación discreta

En la figura 4 se muestra una posible implementación del modelo.

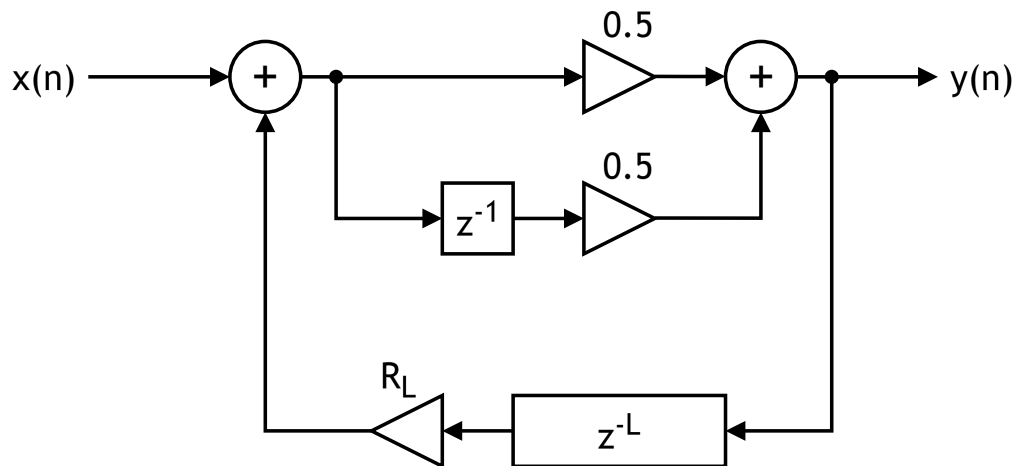


Figura 4: Implementación discreta del modelo

En esta estructura:

- La salida $y(n)$ se toma luego del filtro.
- El filtro implementado es un promedio simple:

$$y(n) = \frac{1}{2}(x(n) + x(n-1))$$

- La frecuencia de resonancia resulta:

$$f_0 \approx \frac{f_s}{L + \frac{1}{2}}$$

- La ganancia R_L controla el tiempo de decaimiento de la nota:
 - $R_L \approx 1$: Decaimiento lento (nota sostenida).
 - $R_L \ll 1$: Decaimiento rápido.

Modelo modificado (síntesis de percusión)

Es posible extender el modelo para generar sonidos más cercanos a instrumentos de percusión. Esto se logra introduciendo una inversión aleatoria de signo en la realimentación, como muestra la figura 5.

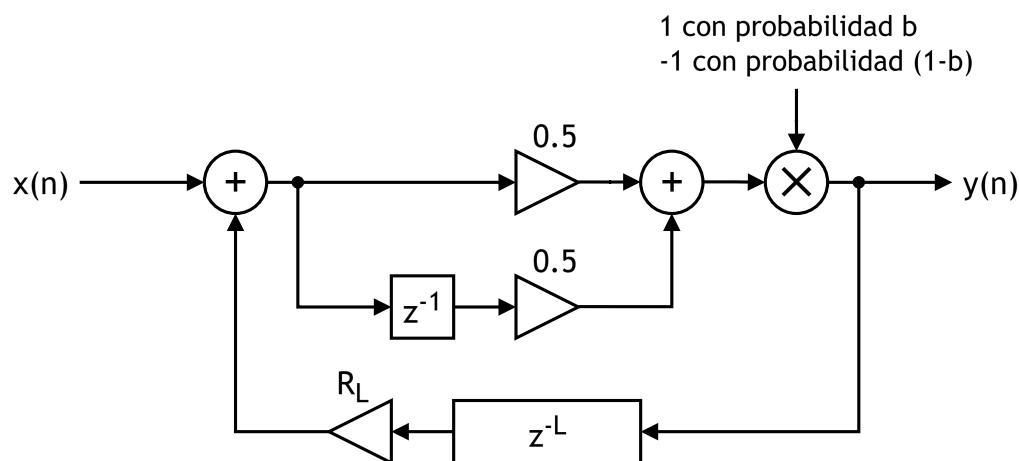


Figura 5: Modelo modificado

En este caso, en cada iteración la señal realimentada es multiplicada por:

$$\text{Ganancia a la salida} = \begin{cases} +1 & \text{con probabilidad } b \\ -1 & \text{con probabilidad } 1 - b \end{cases}$$

Efecto del parámetro b

- $b = 1$: Comportamiento idéntico al modelo original (cuerda).
- $b \approx 0.5$: Sonido más ruidoso, con menor percepción de tono definido y similar a percusión.
- $b = 0$: Inversión sistemática de fase, generando efectos de cancelación y cambios espectrales significativos.

Importante: El sistema modificado no es invariante en el tiempo, por lo que no admite una función de transferencia $H(z)$ en sentido estricto.

Se pide implementar y analizar ambos modelos (original y modificado):

- a) Derivar la función de transferencia $H(z)$ del modelo original y obtener la respuesta en frecuencia.
- b) Determinar y graficar los polos y ceros del sistema.
- c) ¿Cual es el rango de valores que puede tomar R_L para que el sistema sea estable?
- d) Implementar el modelo en la aplicación. Generar señales de salida variando:

- i) L (frecuencia)

- ii) R_L (decaimiento)

- iii) Tipo de ruido inicial (uniforme vs. normal)

Analizar perceptualmente y mediante espectrogramas los resultados obtenidos.

- e) Diseñar e implementar un filtro más realista que emule la caja de resonancia de un instrumento (por ejemplo, una guitarra).

Justificar el diseño propuesto.

- f) Problema de afinación: Dado que L es entero, no es posible obtener cualquier frecuencia deseada. Este error aumenta para frecuencias altas.

- i) Explicar el origen del problema

- ii) Proponer e implementar una solución (por ejemplo, retardo fraccional)

- g) Para el modelo modificado, analizar el efecto del parámetro b sobre el espectro de la señal.

- h) Obtener la fase efectiva del sistema en función de b (considerando un análisis promedio o equivalente).

- i) A partir de la relación:

$$\beta = \frac{f_s}{f_k} - \left(L + \frac{1}{2}\right)$$

encontrar una expresión para b que permita ajustar la frecuencia fundamental deseada.

- j) Discutir las limitaciones de este método:

- i) Distorsión introducida

- ii) Rango de frecuencias útil

Nota final: Con modificaciones adecuadas (filtros más complejos, retardos fraccionales, excitaciones no estacionarias), el modelo de Karplus-Strong puede producir síntesis de muy alta calidad.

Se alienta a explorar mejoras adicionales más allá de lo requerido.

2.3 [OPCIONAL] Síntesis aditiva de sonidos

La síntesis aditiva es una técnica de generación de señales que consiste en construir un sonido como la suma de múltiples señales senoidales de distintas frecuencias (llamadas “parciales” o *partials*), con distinta amplitud. Este modelo se basa en que muchos sonidos musicales pueden describirse como combinaciones de componentes armónicas. La ventaja de la síntesis aditiva es la baja complejidad de cómputo y moderados requerimientos en memoria.

El objetivo será sintetizar instrumentos utilizando la síntesis aditiva, aumentando progresivamente la complejidad del modelo y evaluando para cada uno la calidad perceptual y los requerimientos de cómputo y memoria. La aplicación debe incluir cada una de las implementaciones para poder comparar su calidad perceptual empíricamente.

Para poder llevar a cabo esta tarea y responder las siguientes consignas, obtenga una muestra de audio de una nota sostenida de cuatro (o más) instrumentos (por ejemplo: piano, guitarra, flauta, violín). Esto le permitirá realizar un análisis del contenido espectral y su evolución temporal.

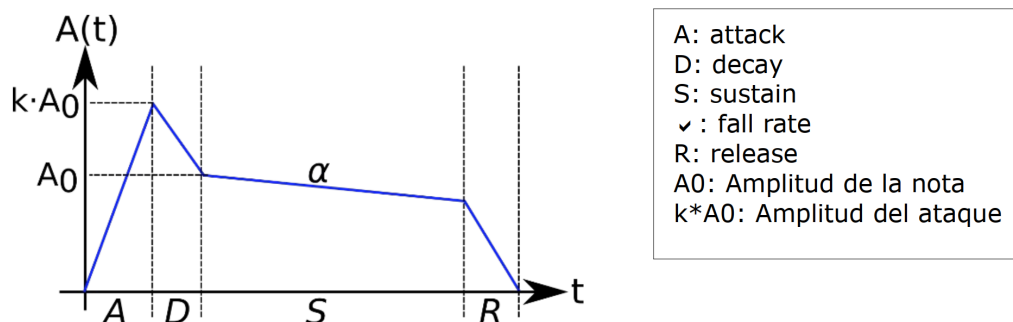
a) Síntesis aditiva sin envolvente

- I) Para cada grabación obtenida, calcular la FFT, utilizando una ventana adecuada, sobre el segmento del audio en la parte sostenida de la nota (segmento estacionario).
- II) Hallar las frecuencias (f_k) en donde se observan los picos y sus respectivas amplitudes (A_k), relativas al primer armónico.
- III) Implementar el modelo de amplitudes constantes y comparar con el audio original:

$$x(t) = \sum_{k=1}^K A_k \sin(2\pi f_k t)$$

b) Síntesis con envolvente temporal ADSR

Como en la práctica cada nota posee una envolvente de amplitud, un modelo más adecuado auditivamente incorpora esta evolución temporal en la amplitud. La manera más básica de modelar la envolvente se denomina ADSR:



- I) A partir de la grabación de cada instrumento, hallar la envolvente de la señal original. Investigue al menos dos métodos para obtener esta envolvente.
- II) A partir de la envolvente de la señal, estime y ajuste los parámetros del modelo ADSR utilizando los criterios que considere necesarios.
- III) Implementar el modelo con envolvente ADSR y comparar con el modelo anterior en cuanto a calidad perceptual y requerimientos de cómputo y memoria.

c) Mejora del modelo de envolvente temporal

Proponer una mejor parametrización y/o forma de envolvente. Por ejemplo:

- Envolventes exponenciales
- Múltiples segmentos
- Interpolación por puntos
- Envolvente extraída directamente de la señal

Implementar el nuevo modelo y comparar con ADSR en cuanto a calidad perceptual y requerimientos de cómputo y memoria.

d) Envolventes por parcial

En general, no es realista suponer que todos los parciales comparten la misma envolvente.

- I) Para cada grabación obtenida, obtener un espectrograma adecuado para observar la evolución temporal de los parciales.
- II) Utilizar la parametrización que considere mejor para modelar la envolvente de cada parcial ($A_k(t)$). Tener en cuenta que la cantidad de parámetros a determinar puede llegar a ser muy grande.
- III) Implementar el modelo y comparar con los modelos anteriores en cuanto a calidad perceptual y requerimientos de cómputo y memoria.

$$x(t) = \sum_{k=1}^K A_k(t) \sin(2\pi f_k t)$$

e) Mejoras del modelo

Investigar e implementar las siguientes mejoras:

- I) Dependencia con el pitch. Analizar la ley de caída de los parciales según la altura (pitch) de la nota y proponer un modelo que tenga en cuenta esta variación.
- II) Desafinación de parciales (detuning). La desviación aleatoria en frecuencia de cada parcial puede transformar el espectro en inarmónico. Estudiar empíricamente el rango apropiado de desviación para “mejorar” el realismo del sonido generado.

2.4 [OPCIONAL] Síntesis de sonidos mediante modulación en frecuencia (FM)

La síntesis por modulación en frecuencia (FM) permite generar señales complejas a partir de la modulación de la fase de una portadora.

Una señal FM puede expresarse como:

$$x(t) = A(t) \cdot \cos(\psi(t))$$

donde $A(t)$ es la envolvente de amplitud y $\psi(t)$ es la fase instantánea.

La frecuencia instantánea está dada por:

$$f_i(t) = \frac{1}{2\pi} \cdot \frac{d}{dt} \psi(t)$$

En síntesis FM, la fase se define como:

$$\psi(t) = 2\pi f_c t + I(t) \cdot \cos(2\pi f_m t + \phi_m) + \phi_c$$

donde:

- f_c es la frecuencia de la portadora,
- f_m es la frecuencia del modulador,
- $I(t)$ es el índice de modulación (posiblemente dependiente del tiempo),
- ϕ_c y ϕ_m son fases iniciales.

Por lo tanto, la señal resultante es:

$$x(t) = A(t) \cdot \cos(2\pi f_c t + I(t) \cdot \cos(2\pi f_m t + \phi_m) + \phi_c)$$

En muchos casos prácticos se utilizan:

$$\phi_m = \phi_c = -\frac{\pi}{2}$$

La calidad del sonido sintetizado depende fundamentalmente de la elección de la envolvente $A(t)$ y del índice de modulación $I(t)$.

Se pide:

a) Implementar un sintetizador FM capaz de generar el sonido de un clarinete. Para ello:

- i) Definir funciones $A(t)$ e $I(t)$ apropiadas, investigando la literatura.
- ii) Utilizar una relación entre frecuencias $f_c/f_m = n/m$, donde n y m son enteros. La frecuencia fundamental resultante será $f_0 = \text{gcd}(f_c, f_m)$.
- iii) Investigar en la literatura valores típicos de n , m y la evolución temporal de $I(t)$ para este instrumento.

Se evaluará el realismo del sonido obtenido.

b) Analizar y sacar conclusiones sobre los siguientes puntos:

- i) Cómo influye $I(t)$ en el contenido espectral.
- ii) El efecto de la relación f_c/f_m en el timbre.
- iii) Limitaciones del modelo FM para la síntesis de instrumentos reales.

3 [OPCIONAL] Efectos de Audio

3.1 Reverberación

a) Implementar al menos dos efectos para aplicar a la salida de la síntesis⁴:

- Eco simple.
- Reverberador plano.
- Reverberador pasa-bajos.
- Reverberador completo.
- Reverberador por convolución. Consiste en convolucionar la señal con la respuesta al impulso de un ambiente reverberante.

Para cada caso:

- Permitir al usuario ajustar los coeficientes o, mejor, proveer ajustes amigables que traduzcan en coeficientes para los filtros.
- Proveer presets.

3.2 Flanger

Un flanger⁵ es un sistema lineal variable en el tiempo. Suma una señal a la misma señal retrasada. El retardo varía lentamente con el tiempo, de manera cíclica. Existen flangers feedforward y feedback. Ver bibliografía citada.

- a) Encontrar la respuesta en frecuencia (en función del tiempo) del flanger.
- b) Implementar al menos dos efectos flanger para aplicar a la salida de la síntesis:
- Vibrato
 - Chorus
 - Flanger
 - Phaser
 - Wah-Wah

3.3 Otros efectos

Se valorará la inclusión de otros efectos de audio (por ejemplo: distorsión, bitcrusher, etc.).

⁴https://ccrma.stanford.edu/~jos/pasp/Artificial_Reverberation.html

⁵<https://ccrma.stanford.edu/~jos/pasp/Flanging.html>

Referencias

- [1] J. O. Smith III, *Global JOS Index*, Center for Computer Research in Music and Acoustics (CCRMA), Stanford University. [Online]. Disponible en: <https://ccrma.stanford.edu/~jos/GlobalJOSIndex.html>
- [2] Aalto University, *Virtual Analog Synthesis and Audio Effects*, Department of Signal Processing and Acoustics. [Online]. Disponible en: <https://www.aalto.fi/en/departement-of-signal-processing-and-acoustics/virtual-analog-synthesis-and-audio-effects>
- [3] J. O. Smith III, *Physical Audio Signal Processing*, W3K Publishing, 2010. [En línea]. Disponible en: <https://ccrma.stanford.edu/~jos/pasp/>
- [4] U. Zölzer (ed.), *DAFX: Digital Audio Effects*, 2nd ed., John Wiley & Sons, 2011.

CONSIDERACIONES

- **Entrega:** 13 de Mayo del 2026, 23:59:59 Hs, o antes. Los trabajos entregados fuera de término serán computados como desaprobados, y regirán las condiciones estipuladas reglamentariamente. Todos los trabajos deben entregarse completos para mantener la condición regular de cursada.
- La evaluación del trabajo práctico será en función del informe escrito presentado.
- Complementariamente a la entrega del informe se podrá requerir una instancia de defensa oral y la reproducción y verificación empírica del trabajo realizado.
- Es condición de **aprobación** responder a los incisos obligatorios, pudiéndose requerir una re-entrega en caso de haber errores o considerarse incompleto. Los ejercicios **opcionales** no son condición necesaria de aprobación.